

Rapport Final

Application de Gestion de Bibliotheque Universitaire

Equipe de Developpement — Universite Norbert Zongo (UNZ)

Cours : Genie Logiciel — Mai 2026

GitHub : <https://github.com/moumounibanao0-afk/bibliotheque-univ-nz1>

1. Introduction

1.1 Contexte

L'Universite Norbert Zongo (UNZ) souhaitait moderniser sa bibliotheque en developpant une application web complete de gestion documentaire. Ce projet s'inscrit dans le cadre du cours de Genie Logiciel et vise a mettre en pratique les concepts etudies : Scrum, Design Patterns, SOLID, tests unitaires et deploiement cloud.

1.2 Objectifs

- Concevoir une application web professionnelle de gestion de bibliotheque universitaire
- Appliquer la methodologie Agile Scrum avec GitHub Projects
- Mettre en oeuvre les principes SOLID et 4 Design Patterns
- Produire une application testee avec couverture $\geq 65\%$
- Deployer l'application sur Render.com

1.3 Perimetre

L'application couvre : gestion des ouvrages, emprunts, reservations, penalites, utilisateurs et statistiques pour 3 acteurs : Etudiant, Bibliothecaire, Administrateur.

2. Analyse des Exigences

2.1 Acteurs

Acteur	Role
Etudiant	Consulte catalogue, emprunte directement, reserve, consulte historique, recoit notifications
Bibliothecaire	Gere catalogue, emprunts, retours, penalites, notifications
Administrateur	Gere utilisateurs, statistiques, configurations et rapports

2.2 Exigences Fonctionnelles

- Gestion du catalogue (ajout, modification, suppression d'ouvrages avec categories)
- Recherche avantee par titre, auteur, ISBN, mots-cles, categorie
- Emprunt direct par l'etudiant et retour d'ouvrages
- Reservation quand tous les exemplaires sont empruntes
- Penalites de retard (100 FCFA/jour standard, 50 FCFA/jour boursier)
- Notification email automatique (confirmation, rappel J-2, disponibilite)
- Historique des emprunts par l'etudiant
- Gestion utilisateurs (inscription avec INE @unz.bf, authentication, 3 roles)
- Statistiques et rapports (graphiques par mois et categorie)
- Gestion des categories et rayons

2.3 Exigences Non Fonctionnelles

Exigence	Valeur
----------	--------

Securite	Authentification JWT, protection des donnees
Performance	Temps de reponse < 2 secondes
Disponibilite	>= 99%
Interface	Responsive (ordinateur et mobile)
Maintenabilite	Principes SOLID, 4 Design Patterns
Versionnement	Git + GitHub

2.4 User Stories

ID	User Story	Acteur	Priorite
US01	S'inscrire avec INE et email @unz.bf	Etudiant	Haute
US02	Se connecter de facon securisee (JWT)	Tous	Haute
US03	Ajouter/modifier un ouvrage avec categorie	Bibliothecaire	Haute
US04	Consulter le catalogue avec categories	Etudiant	Haute
US05	Rechercher par titre, auteur, ISBN, categorie	Etudiant	Haute
US06	Emprunter directement depuis le catalogue	Etudiant	Moyenne
US07	Enregistrer un retour d'ouvrage	Bibliothecaire	Moyenne
US08	Reserver un ouvrage indisponible	Etudiant	Moyenne
US09	Voir les penalites de retard en FCFA	Bibliothecaire	Moyenne
US10	Consulter historique des emprunts	Etudiant	Moyenne
US11	Recevoir emails de notification	Etudiant	Basse
US12	Gerer les utilisateurs (3 roles)	Administrateur	Basse
US13	Voir statistiques et graphiques	Administrateur	Basse
US14	Gerer les categories et rayons	Bibliothecaire	Basse
US15	Generer des rapports	Administrateur	Basse

3. Modelisation et Conception

3.1 Diagramme de Cas d'Utilisation

Le diagramme presente les interactions entre les 3 acteurs et les fonctionnalites : s'inscrire, se connecter, gerer le catalogue, emprunter/retourner, reserver, consulter statistiques. Realise avec PlantUML.

3.2 Diagramme de Classes

- Utilisateur (classe mere) : id, nom, prenom, email, motDePasse, role, dateInscription
- Etudiant (herite de Utilisateur) : numeroEtudiant (INE), filiere, niveau
- Bibliothecaire (herite de Utilisateur) : matricule, service
- Ouvrage : id, titre, auteur, ISBN, nombreExemplaires, exemplairesDisponibles
- Categorie : id, nom, description, rayon
- Emprunt : id, dateEmprunt, dateRetourPrevue, dateRetourReelle, statut
- Reservation : id, dateReservation, statut (EN_ATTENTE/CONFIRMEE/ANNULEE)

3.3 Diagrammes de Sequence

Connexion : Utilisateur saisit identifiants → Controller → Service verifie en base → retourne token JWT 24h.

Emprunt : Etudiant clique Emprunter → Controller verifie disponibilite → Service cree emprunt → decremente exemplaires → email confirmation.

Reservation : Etudiant reserve ouvrage indisponible → Service verifie pas de doublon → cree reservation → email confirmation.

3.4 Architecture 4+1 Vues

Vue	Description
Logique	4 couches : Presentation (HTML/CSS/JS), Application (Controllers), Domaine (Services), Infrastructure
Developpement	Packages : controller, service, repository, model, config, strategy, factory, observer
Processus	Flux : HTTP → Spring Security (JWT) → Controller → Service → Repository → MySQL
Physique	Navigateur → Serveur Spring Boot (port 8081) → MySQL (FreeSQLDatabase)
Scenarios	15 User Stories couvrant 3 acteurs sur 3 sprints

4. Architecture et Design Patterns

4.1 Principes SOLID

Principe	Application
S — Single Responsibility	OuvrageService gere uniquement les ouvrages, EmpruntService les emprunts, NotificationService les
O — Open/Closed	Pattern Strategy : ajout nouveau calcul penalite sans modifier le code existant
L — Liskov Substitution	Etudiant, Bibliothecaire, Admin heritent de Utilisateur et peuvent le substituer
I — Interface Segregation	Interfaces specifiques : Observable, Observateur, CalculPenalite
D — Dependency Inversion	Spring @Autowired : Controllers dependent des interfaces Service, pas des implementations

4.2 Design Patterns

Pattern 1 — Singleton : DatabaseConfig (@Configuration) est geree comme Singleton par Spring IoC. Une seule instance dans le contexte applicatif.

Pattern 2 — Observer : OuvrageDisponibleObservable notifie automatiquement les etudiants via email quand un ouvrage reserve devient disponible apres un retour.

Pattern 3 — Factory : UtilisateurFactory cree le bon type d'utilisateur (Etudiant, Bibliothecaire, Admin) selon le role. Centralise la creation d'objets.

Pattern 4 — Strategy : Interface CalculPenalite avec PenaliteStandard (100 FCFA/jour) et PenaliteBoursier (50 FCFA/jour). Changement d'algorithme sans modifier le code.

5. Implementation et Tests

5.1 Stack Technique

Composant	Technologie
Langage	Java 21
Framework	Spring Boot 3.2.0
Base de donnees	MySQL 8.4
ORM	Hibernate / JPA
Securite	JWT + Spring Security
Frontend	HTML5 / CSS3 / JavaScript
Build	Maven 3.9
Tests	JUnit 5 + Mockito + JaCoCo
Deploiement	Docker + Render.com + FreeSQLDatabase

5.2 Tests Unitaires

Classe de test	Nb tests	Description
OuvrageServiceTest	7	Tests CRUD ouvrages
OuvrageServiceAvanceeTest	7	Tests recherche avancee

EmpruntServiceTest	6	Tests emprunts et retours
ReservationServiceTest	5	Tests reservations
NotificationServiceTest	4	Tests emails
ModelTest	7	Tests modeles JPA
CategorieTest	6	Tests categories
FactoryTest	4	Tests Pattern Factory
AppTest	1	Test demarrage
TOTAL	47	0 echec — 0 erreur

5.3 Couverture de Code (JaCoCo)

Metrique	Valeur
Couverture globale	74.8%
Objectif requis	>= 65%
Resultat	Objectif depasse de 9.8 points
Tests	47 tests unitaires — 0 echec

6. Gestion Agile (Scrum)

6.1 Sprints realises

Sprint 1 — Authentification + Catalogue : US01 inscription INE, US02 connexion JWT, US03 gestion ouvrages, US04 catalogue, US05 recherche avancee. ✓

Sprint 2 — Emprunts + Reservations + Penalites : US06 emprunt direct etudiant, US07 retour, US08 reservation, US09 penalites FCFA, US10 historique. ✓

Sprint 3 — Notifications + Statistiques + Deploiement : US11 emails automatiques, US12 gestion utilisateurs, US13 statistiques graphiques, US14 categories, US15 rapports. ✓

6.2 GitHub Projects

URL : <https://github.com/users/moumounibanao0-afk/projects/12>

Contenu : US01-US15 (Done), Daily Scrum Sprint 1-2-3 (Done), Sprint Review Sprint 1-2-3 (Done), Sprint Retrospective Sprint 1-2-3 (Done), US15 (In Progress).

6.3 Definition of Done

- Code compile sans erreur
- Tests unitaires ecrits et passent ($\geq 65\%$ couverture)
- Code commite sur GitHub avec message descriptif
- Fonctionnalite demontree dans l'interface
- Securite JWT verifiee pour les endpoints proteges

7. Difficultes et Lecons Apprises

7.1 Difficultes

Spring Security bloquait les requetes API : Configuration SecurityFilterChain avec regles precises par role.

Frontend envoyait categorield mais backend attendait objet Categorie : Modification controleur pour accepter Map et liaison CategorieRepository.

Render sans MySQL gratuit : Utilisation FreeSQLDatabase.com avec variables `SPRING_DATASOURCE_URL/USERNAME/PASSWORD`.

Conflits de port 8081 : Utilisation `fuser -k 8081/tcp` avant chaque lancement.

Doublons d'utilisateurs lors des tests : Reinitialisation `AUTO_INCREMENT` et suppression avec `FOREIGN_KEY_CHECKS=0`.

7.2 Lecons Apprises

- Bien configurer l'environnement de developpement des le debut
- Scrum facilite la livraison incrementale et la gestion des priorites
- Les tests unitaires detectent rapidement les regressions
- La documentation des API est essentielle pour la maintenance
- Le versionnement Git avec messages descriptifs ameliore la tracabilite

8. Conclusion

Ce projet a permis de developper une application web complete de gestion de bibliotheque universitaire en appliquant les meilleures pratiques du Genie Logiciel.

Metrique	Valeur
Classes Java	33+
Endpoints API	15+
Tests unitaires	47 (0 echec)
Couverture code	74.8% (objectif 65%)
Design Patterns	4 (Singleton, Observer, Factory, Strategy)
User Stories	15/15 livrees
Sprints	3 sprints de 2 semaines
Technologies	Java 21, Spring Boot 3.2, MySQL 8.4, JWT, Docker

La methodologie Scrum a ete appliquee avec succes. L'application est fonctionnelle localement (<http://localhost:8081>) et deployee sur Render.com. Le code source est disponible sur GitHub avec Product Backlog, tests et documentation completes.